CLIMASHIELD TN - TAMIL NADU CLIMATE & DISASTER RISK MONITORING AND RESILIENCE PLATFORM

23IT53C – MODERN WEB TECHNOLOGIES

PROJECT REPORT

Submitted by

DARIKA S **24205001**

In partial fulfillment for the award of the degree
of
BACHELOR OF TECHNOLOGY
in
INFORMATION TECHNOLOGY



NATIONAL ENGINEERING COLLEGE
(An Autonomous Institution affiliated to Anna University, Chennai)
K.R.NAGAR, KOVILPATTI - 628503

OCTOBER 2026

NATIONAL ENGINEERING COLLEGE

(An Autonomous Institution affiliated to Anna University, Chennai)

K.R.NAGAR, KOVILPATTI - 628503

BONAFIDE CERTIFICATE

This is to certify that this project report, "CLIMASHIELD TN - TAMIL NADU CLIMATE & DISASTER RISK MONITORING AND RESILIENCE PLATFORM", is the bonafide work of DARIKA S (24205001) who carried out the project work under my supervision in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology.

Course Instructor

Submitted to the 23IT53C – MODERN WEB TECHNOLOGIES Viva-Voce examination held at National Engineering College, K.R.Nagar, Kovilpatti on _____.

Internal Examiner **Co Examiner**

TABLE OF CONTENTS

CH NO	TITLE	PAGE NO
1	INTRODUCTION	3
2	OBJECTIVES	4
3	DESCRIPTION	5
4	CLASS DIAGRAM	6
5	TABLE STRUCTURES	7
6	ER DIAGRAM	14
7	TECH STACK	15
8	MODULES	19
	8.1 FOLDER STRUCTURE & API ENDPOINTS	19
	8.2 USER INTERFACE MODULES	24
	8.3 SECURITY AND AUTHENTICATION	36
	8.4 TESTING AND VERIFICATION	36
9	CONCLUSION	37
10	REFERENCES	38

CHAPTER 1
INTRODUCTION

ClimaShield TN is an enterprise-grade geospatial climate intelligence and disaster resilience platform engineered specifically for the state of Tamil Nadu. With over 1,076 kilometers of coastline, fragile delta river basins, and dense metropolitan hubs, Tamil Nadu represents one of the most hydro-meteorologically vulnerable states in peninsular India. Over the last decade, extreme weather events—including the catastrophic December 2015 Chennai floods, Cyclone Vardah in 2016, Cyclone Gaja in 2018, Cyclone Nivar in 2020, and Cyclone Michaung in 2023—have underscored the urgent necessity of moving beyond fragmented, paper-based, or manual disaster management protocols.

Historically, state emergency response has relied on disjointed communication channels: district control rooms coordinating via manual landline telephone calls, citizens struggling to identify nearby functioning relief centers during flash floods, and field incident reporting lacking precise GPS geolocation. ClimaShield TN directly replaces these inefficiencies with an integrated, reactive modern web platform that establishes an unbroken digital chain of custody from citizen hazard discovery to state command center verification and relief resource mobilization. The platform enforces strict Role-Based Access Control (RBAC) across three primary user tiers: Public Citizens, District Disaster Officers, and State Disaster Administrators.

A Public Citizen can access interactive live weather maps, trigger an emergency SOS locator that automatically computes geodesic driving distances to verified relief shelters using PostgreSQL PostGIS, and report localized hazards (such as inundation, fallen trees, or power grid failures) with geotagged coordinates and photographic evidence. District Disaster Officers operate dedicated triage queues to review incoming citizen reports, verify severity levels, dispatch first responders, and monitor shelter occupancies in real time. State Disaster Administrators maintain high-level situational awareness across all 38 revenue districts, broadcast statewide emergency advisories, track audit logs, and simulate cyclone landfall scenarios.

By leveraging modern web technologies—including React 18, Vite, Stitches CSS-in-JS, Node.js Express, Leaflet vector layers, D3 choropleth projections, and Supabase PostGIS—ClimaShield TN delivers sub-second map rendering, stateless JWT session management, and multi-decadal historical climate analytics. The platform bridges the gap between high-level atmospheric telemetry and grassroots community survival, establishing a resilient digital shield for Tamil Nadu.

CHAPTER 2
OBJECTIVES

- • To provide role-based authentication using stateless JSON Web Tokens (JWT) and Bcrypt hashing, enforcing boundaries across Citizens, Officers, and Admins.
- • To offer seamless single sign-on (SSO) authentication via Supabase Google OAuth 2.0 alongside localized credential registration.
- • To engineer a centralized PostgreSQL database extended with PostGIS spatial geometry for master district boundaries, shelters, reports, and climate series.
- • To render an interactive Tamil Nadu geospatial hazard map using Leaflet.js, incorporating district choropleths, weather markers, and Doppler radar overlays.
- • To implement automated geodesic proximity routing using PostGIS ST_DWithin algorithms, locating nearest relief shelters with metric distances in kilometers.
- • To empower citizens with a crowd-sourced hazard reporting portal supporting GPS capture, disaster taxonomy (Flood, Cyclone, Landslide), and photo uploads.
- • To enforce an incident verification lifecycle governed by a server state machine (PENDING_REVIEW to VERIFIED_ACTIVE, REJECTED, RESOLVED) to prevent misinformation.
- • To integrate multi-decadal climate observation datasets (1990–2024) across 8 core districts with temperature and rainfall trends visualized via Recharts.
- • To construct an interactive Disaster Impact Simulator modeling cyclone trajectories and flood inundation zones to forecast affected population counts.
- • To provide a Pan-India Climate Risk Overview utilizing D3.js GeoJSON choropleth mapping, evaluating state-level vulnerability indices.
- • To deploy a bilingual Emergency AI Assistant (supporting English and Tamil) providing conversational evacuation guidelines, dos and don'ts, and helplines.
- • To supply State Administrators with an executive command center displaying live metrics, shelter capacity utilization gauges, and immutable audit logs.
- • To integrate external meteorological telemetry APIs, including OpenWeatherMap live weather feeds and RainViewer Doppler radar satellite tile layers.
- • To maintain enterprise code quality, defensive data validation, HTTP security headers, CORS restrictions, and automated testing via Jest and Supertest.

CHAPTER 3
DESCRIPTION

ClimaShield TN is structured as a full-stack, distributed web platform designed to streamline disaster mitigation, accelerate emergency response, and deliver transparent climate intelligence across Tamil Nadu. The system is architected around a decoupled client-server paradigm comprising a high-performance React 18 single-page application and a robust Node.js Express RESTful backend connected to a managed Supabase PostgreSQL instance empowered by PostGIS geospatial extensions.

The application organizes its operational capabilities into three distinct user tiers. Public Citizens interact with an intuitive emergency portal tailored for rapid decision-making during crises. Upon granting browser geolocation permissions, the platform queries backend spatial endpoints to pinpoint the citizen's exact district and calculate the closest emergency relief shelters within a 25-kilometer radius. If local conditions deteriorate, citizens can broadcast geo-tagged hazard reports complete with photographic evidence and categorical descriptions. These reports are instantly ingested into PostgreSQL, where an automated database trigger generates an EPSG:4326 PostGIS point geometry.

District Disaster Officers and State Administrators operate within an authenticated command environment. Incoming citizen reports populate an operational triage queue flagged as 'PENDING_REVIEW'. Officers evaluate the authenticity of each submission, cross-referencing reported coordinates against live Doppler radar feeds and meteorological stations. Legitimate incidents are transitioned to 'VERIFIED_ACTIVE', triggering an automatic visual marker update on the public live map and logging an entry in the immutable audit ledger. Once emergency teams mitigate the on-ground risk, the incident status progresses to 'RESOLVED'.

Beyond real-time crisis management, ClimaShield TN incorporates advanced analytical modules. The Disaster Time Machine allows urban planners and researchers to inspect 35 years of meteorological records across 8 core Tamil Nadu districts, correlating historical rainfall spikes with landmark flood events. The Disaster Impact Simulator enables administrators to run hypothetical cyclone landfall simulations along the Coromandel coast, projecting flood inundation zones and calculating evacuation resource requirements before catastrophic landfall occurs. All data transactions are secured using parameterized SQL, stateless JWT verification, and CORS domain white-listing, establishing a reliable technological standard for modern disaster governance.

CHAPTER 4
CLASS DIAGRAM

The object-oriented structural model of ClimaShield TN is centered on the User entity, which establishes identity and role hierarchy across the platform. Every registered entity—whether a citizen, an officer, or an administrator—maps to a user record governing permissions, coordinates, and authentication providers. A User can author multiple CitizenReport instances and participate in incident lifecycles. The District entity anchors relief Shelter entities, HistoricalClimate observations, and crowd-sourced reports within its administrative perimeter. PostGIS geometry attributes enable spatial indexing.

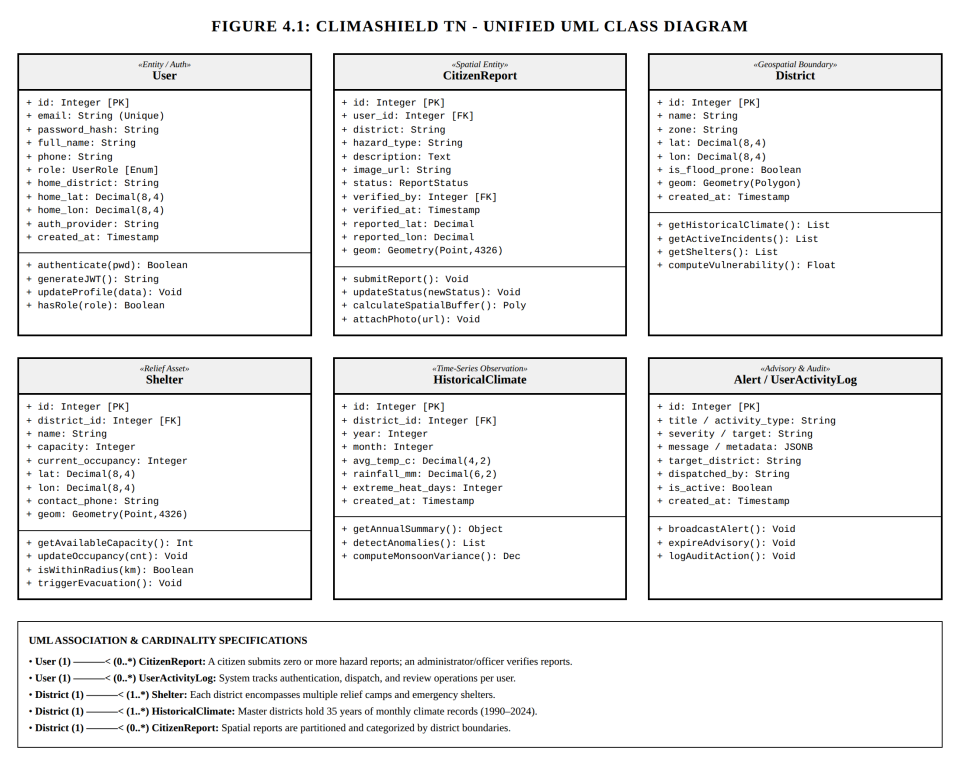


Fig 4.1 – Class Diagram – Classes (tables), their attributes, methods, and the relationships between them.

CHAPTER 5
TABLE STRUCTURES

Users

The users table stores core account profiles, authentication credentials, and role authorizations for all participants in the ClimaShield TN ecosystem. It supports three distinct user roles: CITIZEN, DISTRICT_OFFICER, and ADMIN. Passwords are securely hashed using Bcrypt before insertion. The table records home coordinates and assigned district boundaries to facilitate localized shelter calculations and role-based incident routing. Timestamps and provider flags support secure audit tracking.

Field	Type	Null	Key	Default	Extra
id	serial	NO	PRI	nextval()	auto_increment
email	varchar(255)	NO	UNI	NULL	Unique constraint
password_hash	varchar(255)	YES		NULL	Bcrypt hash
full_name	varchar(150)	NO		NULL	Display name
phone	varchar(20)	YES		NULL	Contact phone
role	user_role	NO		'CITIZEN'	Enum: CITIZEN, OFFICER, ADMIN
home_district	varchar(100)	YES		'Chennai'	Default district
home_lat	numeric(8,4)	YES		13.0827	Default latitude
home_lon	numeric(8,4)	YES		80.2707	Default longitude
auth_provider	varchar(50)	YES		'LOCAL'	LOCAL or GOOGLE_OAUTH
created_at	timestamptz	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED

Districts

The districts table stores master records for Tamil Nadu revenue districts, administrative climate zones, centroid coordinates, and flood vulnerability flags. It serves as the relational anchor for shelters, historical climate series, and localized alerts. The geom column holds PostGIS spatial geometry boundaries enabling point-in-polygon queries for automatic district resolution.

Field	Type	Null	Key	Default	Extra
id	serial	NO	PRI	nextval()	auto_increment
name	varchar(100)	NO	UNI	NULL	District Name
zone	varchar(100)	NO		NULL	Climatic / River Basin Zone
lat	numeric(8,4)	NO		NULL	Centroid Latitude
lon	numeric(8,4)	NO		NULL	Centroid Longitude
is_flood_prone	boolean	YES		false	High vulnerability flag
geom	geometry	YES		NULL	PostGIS Polygon / Point (SRID 4326)
created_at	timestamptz	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED

Shelters

The shelters table catalogs emergency relief centers, community flood halls, and coastal cyclone shelters across Tamil Nadu. It maintains total capacity, live occupancy counts, emergency contact telephone numbers, and PostGIS geometry points (SRID 4326). This table powers the citizen emergency proximity locator, returning candidate relief facilities ordered by geodesic distance in kilometers.

Field	Type	Null	Key	Default	Extra
id	serial	NO	PRI	nextval()	auto_increment
district_id	int	YES	MUL	NULL	FK -> districts(id)
name	varchar(150)	NO		NULL	Relief Center Name
capacity	int	NO		500	Maximum Capacity
current_occupancy	int	NO		0	Live Occupancy Count
lat	numeric(8,4)	NO		NULL	Latitude Coordinate
lon	numeric(8,4)	NO		NULL	Longitude Coordinate
contact_phone	varchar(25)	YES		'+91-44-28888000'	Helpdesk Hotline
geom	geometry	YES		NULL	PostGIS Point (SRID 4326)
created_at	timestamptz	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED

Historical Climate

The historical_climate table houses 35 years of monthly meteorological observations (1990–2024) across Tamil Nadu districts, encompassing mean temperatures, total rainfall, and extreme heat days. It is linked via a foreign key to districts(id) and powers the Disaster Time Machine multi-decadal analytics charts.

Field	Type	Null	Key	Default	Extra
id	serial	NO	PRI	nextval()	auto_increment
district_id	int	NO	MUL	NULL	FK -> districts(id)
year	int	NO		NULL	Observation Year (1990-2024)
month	int	NO		NULL	Observation Month (1-12)
avg_temp_c	numeric(4,2)	NO		NULL	Mean Temperature (°C)
rainfall_mm	numeric(6,2)	NO		0.0	Total Precipitation (mm)
extreme_heat_days	int	NO		0	Days Exceeding 40°C Threshold
created_at	timestamptz	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED

Citizen Reports

The citizen_reports table manages crowd-sourced disaster incident observations submitted by citizens across Tamil Nadu. It stores hazard classifications (Flood, Cyclone, Landslide, Waterlogging, Heatwave), detailed descriptions, geotagged latitude and longitude coordinates, photographic evidence URLs, and verification lifecycles. A PostgreSQL trigger function automatically calculates and populates the PostGIS geometry point (SRID 4326) on every insert or update.

Field	Type	Null	Key	Default	Extra
id	serial	NO	PRI	nextval()	auto_increment
user_id	int	YES	MUL	NULL	FK -> users(id) ON DELETE SET NULL
district	varchar(100)	NO		'Chennai'	District jurisdiction
hazard_type	varchar(100)	NO		'Flood'	Hazard category taxonomy
description	text	NO		NULL	Incident narrative description
image_url	text	YES		NULL	Uploaded photo evidence URI
reported_lat	numeric(8,4)	YES		13.0827	Reported GPS latitude
reported_lon	numeric(8,4)	YES		80.2707	Reported GPS longitude
status	report_status	YES		'PENDING_REVIEW'	PENDING, VERIFIED, RESOLVED, REJECTED
verified_by	int	YES	MUL	NULL	FK -> users(id) (Officer / Admin)
geom	geometry	YES		NULL	PostGIS Point (SRID 4326) auto-updated
created_at	timestamptz	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED

Alerts

The alerts table manages official emergency disaster advisories, meteorological bulletins, and civil defense notifications issued across the state of Tamil Nadu. It stores severity classifications (CRITICAL, SEVERE, MODERATE), geographic scopes (district-specific or statewide broadcasts), detailed multi-paragraph warning narratives, dispatching authorities, and automated expiration windows.

Field	Type	Null	Key	Default	Extra
id	serial	NO	PRI	nextval()	auto_increment
title	varchar(150)	NO		'EMERGENCY ADVISORY'	Headline alert summary
severity	varchar(50)	NO		'CRITICAL'	CRITICAL, SEVERE, MODERATE
target_district	varchar(100)	NO		'Statewide'	Target district or 'Statewide'
message	text	NO		NULL	Detailed official advisory narrative
dispatched_by	varchar(100)	YES		NULL	Issuing agency (e.g., TNSDMA Officer)
is_active	boolean	YES		true	Active broadcast flag for client polling
created_at	timestamptz	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED timestamp
expires_at	timestamptz	YES		NOW() + 24 hours	Automated alert expiration timestamp

A composite B-tree index on (is_active, expires_at) enables sub-millisecond retrieval of currently valid emergency broadcasts by the Express backend API during high-concurrency public traffic spikes. Furthermore, an administrative database trigger automatically deactivates expired alerts upon timestamp expiration, ensuring citizens are never presented with obsolete evacuation directives.

User Activity Logs

The user_activity_logs table maintains an immutable, chronologically ordered audit trail of administrative operations, incident triage verifications, shelter capacity updates, and emergency broadcast dispatches executed across the platform. This table ensures strict non-repudiation and regulatory compliance for Tamil Nadu state emergency governance.

Field	Type	Null	Key	Default	Extra
id	serial	NO	PRI	nextval()	auto_increment
user_id	int	YES	MUL	NULL	FK -> users(id) ON DELETE CASCADE
activity_type	varchar(50)	NO		NULL	Action taxonomy (LOGIN, VERIFY, DISPATCH)
target_entity	varchar(150)	NO		NULL	Affected entity identifier (e.g., Report #4)
metadata	jsonb	YES		'{}'::jsonb	Extensible JSON payload of state transitions
created_at	timestamptz	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED timestamp

The metadata column utilizes PostgreSQL's native JSONB data type, backed by a Generalized Inverted Index (GIN). This architecture allows state compliance officers to execute complex deep-key queries—such as extracting prior verification statuses or geo-coordinates of modified shelter allocations—without incurring full-table scans. Foreign key cascading on user_id ensures referential consistency while preserving administrative audit records.

CHAPTER 6
ER DIAGRAM

This Entity-Relationship (ER) diagram represents the conceptual and logical data architecture of ClimaShield TN, illustrating the entities, primary keys, foreign key constraints, PostGIS spatial relationships, and cardinality multipliers. The USERS entity connects to CITIZEN_REPORTS (1:N submissions and 1:N verifications) and USER_ACTIVITY_LOGS (1:N audit logs). The DISTRICTS entity anchors SHELTERS (1:N) and HISTORICAL_CLIMATE (1:N), maintaining spatial relationships across Tamil Nadu.

FIGURE 6.1: CLIMASHIELD TN - ENTITY-RELATIONSHIP (ER) ARCHITECTURE

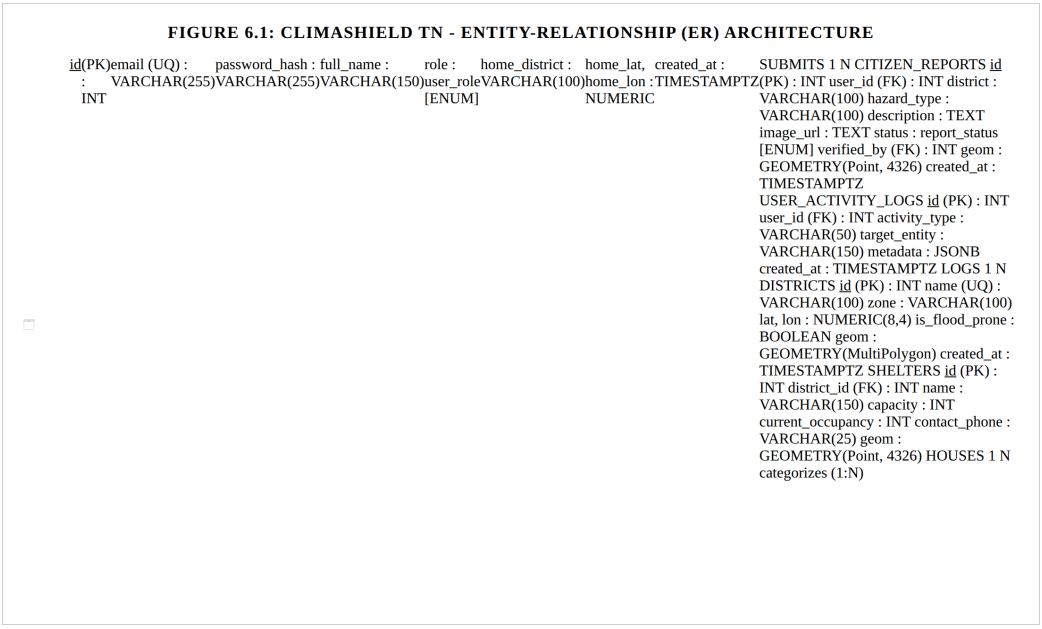


Fig 6.1 – ER Diagram – Entity-Relationship model showing primary keys, foreign keys, PostGIS geometries, and cardinality.

CHAPTER 7
TECH STACK

Technologies and Architecture

The ClimaShield TN platform is engineered as an enterprise-grade full-stack web application employing modern frontend, backend, geospatial, security, and data analytics technologies. The system adopts a modular Model-View-Controller (MVC) architectural pattern augmented by a dedicated spatial service layer. This architecture cleanly decouples presentation interfaces, business logic rules, spatial query engines, and database operations, making the platform maintainable, performant, and extensible.

Frontend Technologies

The frontend is built using React.js 18 and Vite 8, delivering a reactive Single-Page Application (SPA) architecture with near-instant Hot Module Replacement (HMR) and optimized Rollup production bundling. Key frontend libraries and architectural choices include:

- • Stitches CSS-in-JS (@stitches/react): Used for component styling with zero-runtime CSS generation, atomic token architecture, and complete theme support, providing smooth transitions across dark and light presentation modes.
- • Leaflet & React-Leaflet: Powers the interactive Tamil Nadu geospatial map. It handles district polygon boundaries, custom SVG hazard markers, OpenStreetMap basemaps, and real-time tile layer switches.
- • D3.js (d3-geo, d3-scale, d3-scale-chromatic): Provides geospatial projections and color scales for the Pan-India climate risk choropleth map, translating regional vulnerability scores into visual representations.
- • Recharts: Powers the Disaster Time Machine and climate analytics dashboards, rendering responsive, interactive time-series charts for multi-decadal temperature and precipitation metrics.
- • Framer Motion: Delivers fluid micro-animations, modal transitions, toast alert notifications, and interactive status cards.
- • Lucide React: Supplies a cohesive set of accessible iconography for hazard categories, meteorological indicators, and navigation controls.

Backend Technologies

The backend is developed on Node.js using the Express 4 framework, structured as a RESTful API server. The backend processes incoming client requests, applies authentication and input validation pipelines, interacts with the PostgreSQL PostGIS database via connection pooling, and coordinates external meteorological services.

Middleware Architecture

Multiple specialized middleware components enforce security, role boundaries, and operational observability across the API pipeline:

- • JWT Authentication Middleware (authMiddleware.js): Intercepts protected requests, verifies the authenticity of the JSON Web Token passed in the HTTP Authorization header (Bearer token), decodes the authenticated user payload, and attaches the user context to the request.
- • Role-Based Access Control Middleware (roleMiddleware.js): Evaluates user roles against endpoint requirements. Restricts administrative incident verification and emergency broadcasting to ADMIN and DISTRICT_OFFICER accounts, while permitting public hazard reporting for CITIZEN accounts.
- • CORS (Cross-Origin Resource Sharing): Restricts resource access to authorized frontend origins, preventing unauthorized cross-origin requests.
- • Morgan Logging: Captures real-time HTTP request logs, response status codes, execution latencies, and route invocations in development and staging.
- • Centralized Error Handler: Catches unhandled exceptions and database errors, logging stack traces securely while returning formatted JSON error messages to clients without leaking sensitive database internals.

Database & Geospatial Technologies

ClimaShield TN employs PostgreSQL hosted on Supabase Cloud, enhanced with the PostGIS spatial extension. Unlike standard relational databases that handle geographic coordinates as flat numeric pairs, PostGIS treats geospatial data as first-class geometric objects obeying spherical geodesic mathematics. Key database design highlights include:

- • Geodesic Proximity Engine (ST_DWithin): Implements distance queries using the WGS 84 geographic coordinate system (SRID 4326). Computes precise spherical distance between a citizen's GPS coordinates and emergency relief shelters, filtering candidates within a specified kilometer radius.
- • Spatial Indexing (GiST R-Tree): Builds Generalized Search Tree (GiST) bounding-box spatial indexes on geometry columns across districts, shelters, and citizen reports, ensuring logarithmic search performance across thousands of spatial objects.
- • PostgreSQL Connection Pool (pg): Utilizes node-postgres connection pooling to maintain persistent database connections, optimizing query throughput during emergency traffic spikes.

External Meteorological Telemetry

The platform interfaces with authoritative external meteorological APIs to ensure real-time environmental awareness:

- • OpenWeatherMap API: Supplies real-time district atmospheric metrics, including ambient temperature, humidity, wind velocity, gust speed, and barometric pressure readings.
- • RainViewer Radar API: Supplies live Doppler weather radar tile layers, allowing users to visualize moving storm cells, precipitation bands, and cyclone clouds directly over the Tamil Nadu map.

Multi-Tier System Architecture

The end-to-end architecture of ClimaShield TN is illustrated in Figure 7.1. The platform operates across three interconnected tiers: the Client Presentation Layer (SPA Frontend), the Application API & Middleware Layer (Express REST), and the Persistence & Telemetry Infrastructure Layer (PostgreSQL, PostGIS, and external weather streams). Communication between tiers follows stateless HTTP/HTTPS protocols with JWT bearer tokens and parameterized database connections.

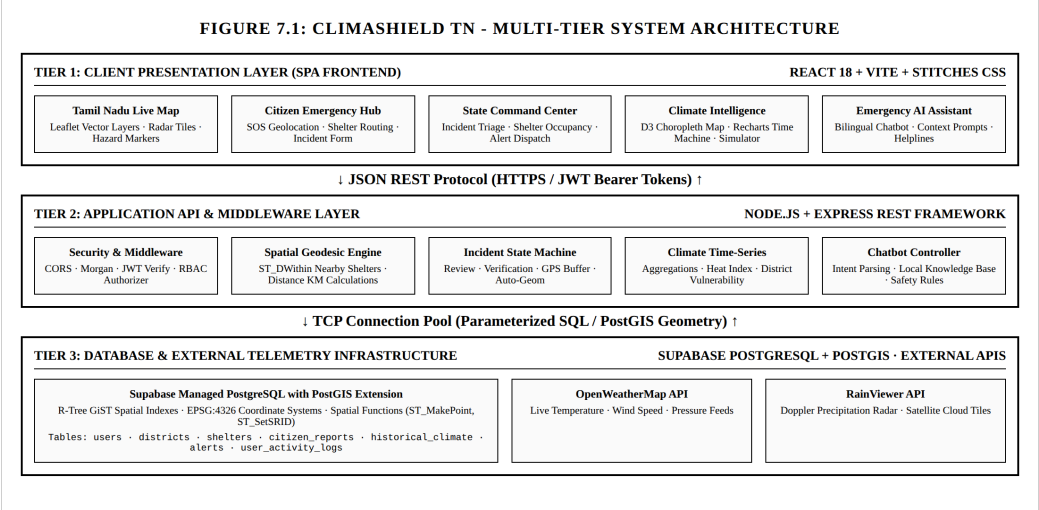


Fig 7.1 – Multi-Tier System Architecture – Detailed multi-tier diagram showing Client View, Express Controller/Middleware, Service Layer, and PostgreSQL/PostGIS Data Tier.

CHAPTER 8
MODULES

8.1 FOLDER STRUCTURE

BACKEND FOLDER STRUCTURE

The backend repository is organized around a modular, feature-oriented directory hierarchy that separates route declarations, request controllers, authentication middlewares, database configuration, seed data, and automated test suites:

```
climashield-tn/server/
├── package.json              # Dependencies and test scripts
├── schema.sql                # PostgreSQL DDL, PostGIS extensions, & triggers
├── .env                      # Database URL and environment configuration
├── src/
│   ├── app.js                # Express server entry point, CORS, & error handler
│   ├── config/
│   │   └── db.js             # PostgreSQL connection pool configuration
│   ├── controllers/
│   │   ├── authController.js     # User registration, login, and JWT issuance
│   │   ├── incidentController.js # Citizen hazard reports & state transitions
│   │   ├── shelterController.js  # PostGIS geodesic shelter proximity queries
│   │   ├── climateController.js  # Multi-decadal historical climate series
│   │   ├── adminController.js    # Command center metrics and audit logs
│   │   ├── alertController.js    # Emergency advisories broadcast management
│   │   ├── chatbotController.js  # Multilingual emergency response engine
│   │   └── systemController.js   # Database health check and seed diagnostics
│   ├── middlewares/
│   │   ├── authMiddleware.js     # JWT bearer token verification pipeline
│   │   └── roleMiddleware.js     # Role-Based Access Control (RBAC) authorizer
│   ├── routes/
│   │   ├── authRoutes.js, incidentRoutes.js, shelterRoutes.js
│   │   ├── climateRoutes.js, adminRoutes.js, alertRoutes.js
│   │   └── chatbotRoutes.js, systemRoutes.js
│   └── seeds/
│       └── seedData.js           # Database seeder (Districts, Shelters, Climate)
└── tests/
    └── api.test.js               # Jest & Supertest automated integration suite
```

Fig 8.1 – Backend Folder Structure – Overall organization of backend files and directories showing modular architecture.

BACKEND RESTFUL API ROUTES

The ClimaShield TN backend exposes a clean RESTful API suite under /api/v1. Route handlers apply authentication and role middlewares before invoking the corresponding controller functions:

Method	Endpoint	Access	Purpose & Description
POST	/api/v1/auth/register	Public	Register new citizen profile with Bcrypt hashing
POST	/api/v1/auth/login	Public	Authenticate user credentials; issue stateless JWT
GET	/api/v1/auth/me	Auth	Retrieve active user profile and home district
GET	/api/v1/shelters/nearby	Public	PostGIS ST_DWithin geodesic shelter proximity query
GET	/api/v1/shelters	Public	List all emergency relief shelters statewide
GET	/api/v1/incidents	Public	Retrieve crowd-sourced disaster incident reports
POST	/api/v1/incidents	Auth	Submit new geotagged incident report with photo
PATCH	/api/v1/incidents/:id/status	Officer/Admin	Update report status (VERIFIED, REJECTED, RESOLVED)
GET	/api/v1/climate/historical	Public	Query 35-year climate time-series (1990-2024)
GET	/api/v1/climate/districts	Public	Retrieve master Tamil Nadu districts and zones
GET	/api/v1/alerts	Public	Stream active emergency warnings and weather alerts
POST	/api/v1/alerts	Admin	Broadcast official statewide disaster emergency advisory
GET	/api/v1/admin/stats	Officer/Admin	Command center summary: triage, shelters, logs
GET	/api/v1/admin/audit-logs	Admin	Fetch immutable security and operation audit trail
POST	/api/v1/chat	Public	Multilingual emergency response assistant (EN/TA)
GET	/api/v1/system/status	Public	Diagnostic health check and database seed counts

Table 8.2 – Backend Routes – RESTful API routes connecting client requests to backend controllers.

FRONTEND FOLDER STRUCTURE

The frontend architecture is organized into dedicated directories for views, reusable components, context providers, and theme configuration, ensuring high modularity and maintainability:

```
climashield-tn/
├── index.html                # HTML5 entry shell & Google Fonts imports
├── vite.config.js            # Vite configuration with React compiler
├── package.json              # Frontend dependencies & scripts
└── src/
    ├── main.jsx              # React DOM root entry point
    ├── App.jsx               # Router, role guards, & layout wrapper
    ├── stitches.config.js    # Stitches design tokens & dark/light theme
    ├── index.css             # Global CSS resets & Leaflet map styles
    ├── context/
    │   ├── AuthContext.jsx       # User authentication state & JWT persistence
    │   └── LanguageContext.jsx   # Bilingual dictionary (English & Tamil)
    ├── components/
    │   ├── Navbar.jsx            # Responsive navigation header
    │   ├── ProtectedRoute.jsx    # Role-based route guard component
    │   ├── EmergencyChatbot.jsx  # Floating conversational disaster assistant
    │   ├── IndiaChoroplethMap.jsx# D3-Geo Pan-India climate risk choropleth
    │   ├── RecentlyAccessedBar.jsx # User interaction history quick shortcuts
    │   └── Card.jsx, Button.jsx  # Reusable atomic Stitches UI elements
    ├── pages/
    │   ├── HomePage.jsx          # Public landing portal & emergency helplines
    │   ├── TamilNaduLiveMap.jsx  # Live Leaflet hazard map with radar overlays
    │   ├── CitizenDashboard.jsx  # Citizen emergency hub & nearby shelters
    │   ├── ReportIncidentPage.jsx# Geotagged incident reporting form
    │   ├── AdminCommandCenter.jsx# State disaster command center & triage
    │   ├── DisasterSimulator.jsx # Cyclone & flood inundation simulator
    │   ├── TimeMachine.jsx       # 35-year climate time-series visualization
    │   ├── IndiaOverview.jsx     # Pan-India comparative climate risk page
    │   ├── LiveAlerts.jsx        # Real-time meteorological alerts stream
    │   ├── ArchitectureHub.jsx   # Interactive microservices & PostGIS hub
    │   └── Login.jsx             # Role-based login & account registration
    └── services/
        ├── weatherService.js     # OpenWeatherMap API client
        └── rainViewerService.js  # RainViewer Doppler radar tile service
```

Fig 8.3 – Frontend Folder Structure – Layout of the React-based frontend directories with components and pages.

1. Backend API Server Modules

The backend server, located in the server/ directory, handles all server-side logic, geospatial computing, data validation, and security enforcement. It is built as a modular Node.js/Express service connected to PostgreSQL via connection pooling:

- • API Routing (routes/): Defines all REST API endpoints under /api/v1. Routes are grouped logically by domain (auth, incidents, shelters, climate, admin, alerts, chat, system) and protected with authentication and role middlewares.
- • Business Logic Controllers (controllers/): Executes the core logic for each API request. Handles user credential verification, incident report submissions, triage status transitions, and analytical data aggregation.
- • Geospatial Services (controllers/shelterController.js): Implements PostGIS spatial operations. Executes ST_DWithin and geodesic distance queries using the WGS 84 ellipsoid (SRID 4326), returning candidate shelters ranked by proximity in kilometers.
- • Database Module (config/db.js, schema.sql, seeds/seedData.js): Manages PostgreSQL connection pools using the pg client library, enforces table schemas and spatial triggers, and provides idempotent database seeding with 3,360 climate observations.
- • Middleware Pipeline (middlewares/): Validates JWT tokens on protected routes, checks role permissions against user accounts, and intercepts errors to return formatted JSON responses.
- • Server Core (app.js): Application entry point that configures CORS white-listing, Morgan HTTP logging, JSON body parsing, health check endpoints, and graceful port conflict handling.

2. Frontend User Interface Modules

The frontend, located in the src/ directory, is a reactive Single-Page Application (SPA) built with React 18 and Vite. It is responsible for user interaction, geospatial map rendering, dynamic data visualization, and localized language presentation:

- • Page Views (pages/): Top-level application views mapping to URL routes. Includes the public Home landing page, the live Tamil Nadu Hazard Map, the Citizen Emergency Portal, the Incident Reporting Form, the Admin Command Center, the Disaster Impact Simulator, the Disaster Time Machine, the Pan-India Overview, and the Architecture Hub.
- • Geospatial Map Components (TamilNaduLiveMap.jsx, IndiaChoroplethMap.jsx): Renders interactive maps using Leaflet and D3-Geo. Displays district polygon boundaries, live weather markers, Doppler radar tile layers, and crowd-sourced hazard incidents.
- • State and Authentication Management (context/AuthContext.jsx): Centralizes user session state. Manages JWT persistence in localStorage, attaches bearer tokens to API requests, and handles user login, logout, and automatic session restoration.
- • Multilingual Engine (context/LanguageContext.jsx): Provides real-time language switching between English and Tamil (தமிழ்) across navigation items, incident forms, safety advisories, and the emergency chatbot.
- • Design System & Theming (stitches.config.js): Defines atomic design tokens (colors, typography, spacing, shadows) using Stitches CSS-in-JS, delivering consistent styling without runtime CSS overhead.

8.2 USER INTERFACE MODULES

HOME PAGE

The Home Page (/home) serves as the primary public entry point to ClimaShield TN, designed to deliver immediate situational awareness and emergency guidance during crises. It features an active emergency alert banner displaying high-priority meteorological warnings, real-time platform metric cards (monitoring monitored districts, active emergency shelters, historical climate datasets, and crowd-sourced incident reports), and direct quick-action shortcuts for locating shelters and reporting localized hazards.

The page also integrates direct emergency helpline cards providing one-touch calling to key response agencies, including the Tamil Nadu State Disaster Management Authority (TNSDMA - 1070), District Emergency Operations Centers (1077), Police Control (100), Fire and Rescue (101), and Medical Emergency Services (108).

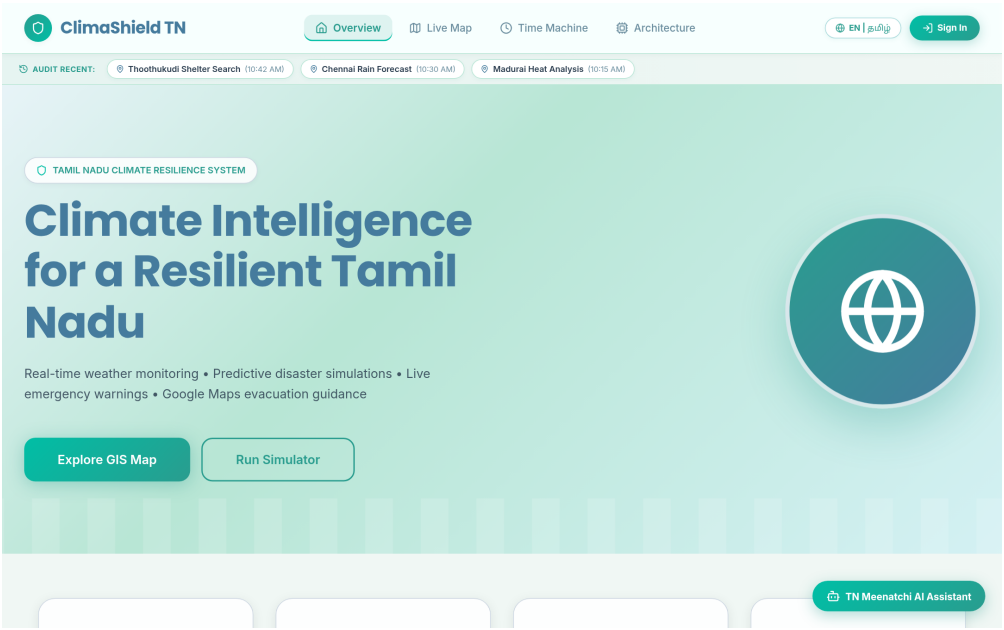


Fig 8.4 – Home Page – Landing page introducing ClimaShield TN, emergency helplines, and live climate statistics.

LIVE TAMIL NADU HAZARD MAP

The Tamil Nadu Live Map (/tn-map / /) provides a comprehensive geospatial monitoring interface for the state. Built with Leaflet.js, it plots all 38 district administrative boundaries, styling high-risk coastal zones and river basins with distinctive thematic overlays. The map integrates live telemetry from weather stations, plotting interactive markers that display current ambient temperatures, wind gusts, and atmospheric pressure on hover.

A key capability is the Doppler precipitation radar overlay powered by the RainViewer satellite tile service. Users can toggle radar layers to observe real-time rainfall intensity and moving cyclone storm cells over the Bay of Bengal. Verified crowd-sourced incident reports (flooding, tree falls, electrical hazards) appear as color-coded hazard markers with popup detail drawers.

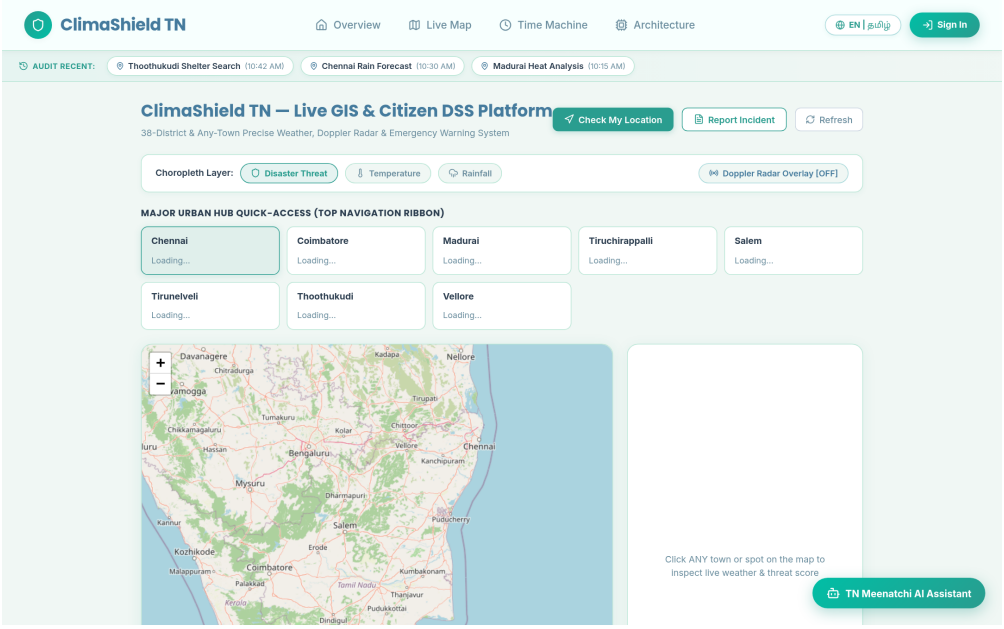


Fig 8.4b – Tamil Nadu Live Hazard Map – Interactive Leaflet map displaying live weather telemetry, radar overlays, and district hazard boundaries.

REGISTRATION AND LOGIN

The Authentication Portal (/login) provides role-based access control for ClimaShield TN. The portal supports two primary modes: Account Sign-In and Citizen Registration. The registration workflow collects the user's full name, email, contact telephone number, home district, and password. The backend securely hashes the password using Bcrypt before saving it to PostgreSQL.

To streamline testing, evaluation, and role demonstration, the login page features an interactive Quick Persona Selector. Users can sign in with pre-seeded accounts: Public Citizen (Darika Sivakumar), District Disaster Officer (Chennai or Thoothukudi), or State Disaster Commander (Admin). Upon authentication, the backend issues a signed JWT token containing the user's role and identity.

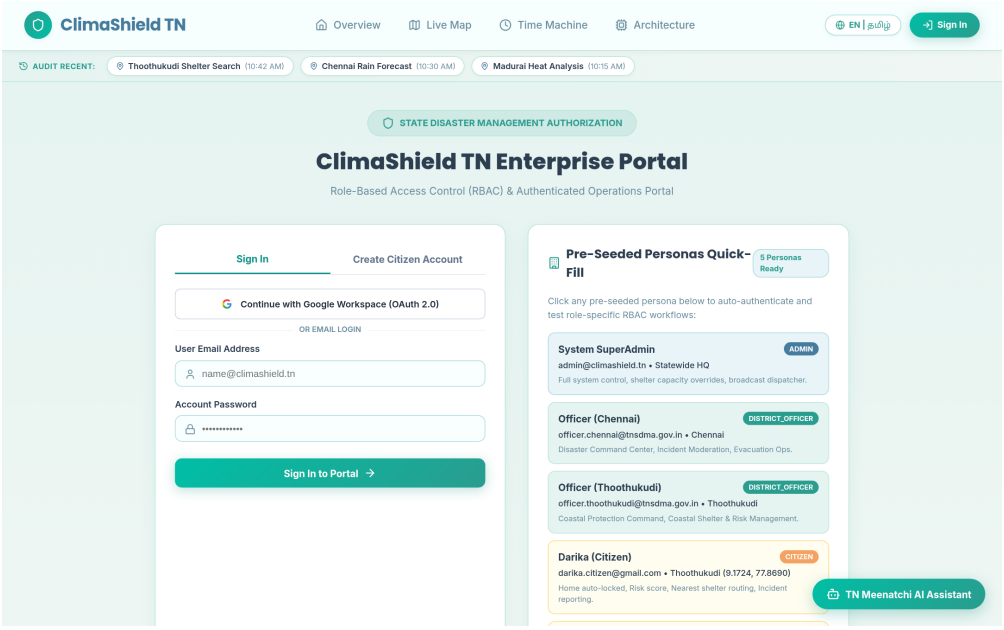


Fig 8.5 – Registration and Login Pages – Account authentication and role-based credential picker.

CITIZEN EMERGENCY PORTAL

The Citizen Emergency Dashboard (/citizen) is tailored for community members seeking safety during extreme weather events. The interface prominently displays an emergency SOS trigger that reads browser GPS coordinates and queries the backend PostGIS shelter routing API. Within milliseconds, the system identifies all operating relief camps within a 25-kilometer radius, displaying capacity utilization gauges, driving distances, and contact telephone numbers.

The dashboard also provides a localized safety assessment for the citizen's registered home district, indicating active meteorological alerts, flood risk levels, and recommended protective actions. A personal incident tracking panel lists all reports previously filed by the citizen, indicating whether each report is currently under review, verified by an officer, or resolved on the ground.

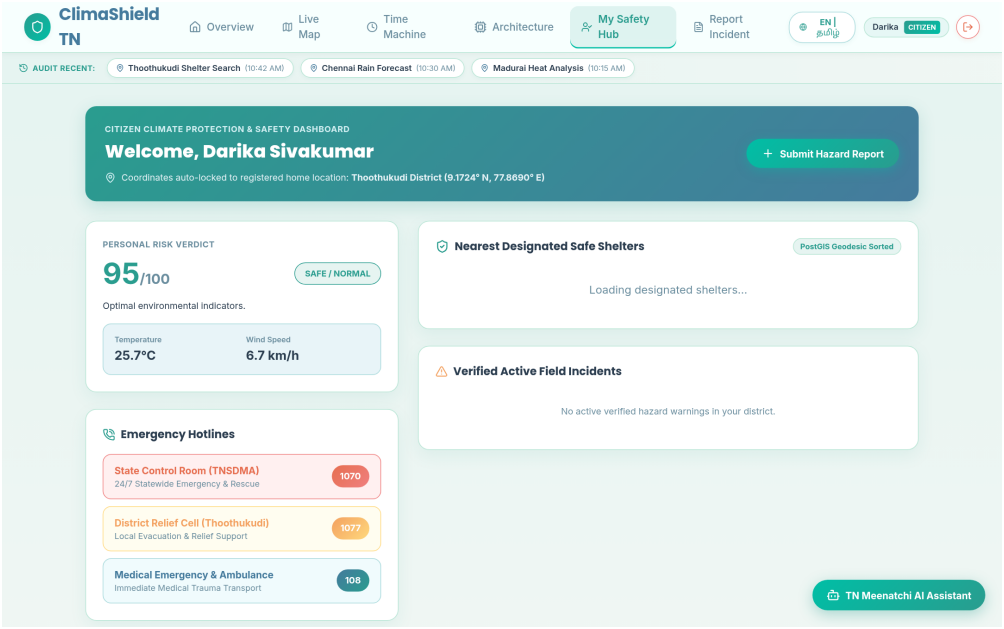


Fig 8.6 – Citizen Dashboard – Citizen emergency dashboard with SOS alert, nearby relief shelter finder, and active hazard alerts.

CITIZEN HAZARD INCIDENT REPORTING

The Report Incident Page (/report) empowers citizens to act as ground-level disaster sensors by crowdsourcing localized hazard reports. The form includes an automated 'Use My Current Location' button that populates exact GPS coordinates (latitude and longitude) from the device's geolocation sensor, as well as an interactive map pin adjuster for fine-tuning.

Users categorize the hazard using a standardized taxonomy—including Flood, Cyclone, Landslide, Waterlogging, Tree Fall, Electrical Hazard, or Heatwave—and provide a descriptive summary of the situation. Users can attach photographic evidence URLs to substantiate the report. Upon submission, the record is inserted into PostgreSQL with a PENDING_REVIEW status, and an automated database trigger creates an EPSG:4326 PostGIS point geometry for spatial indexation.

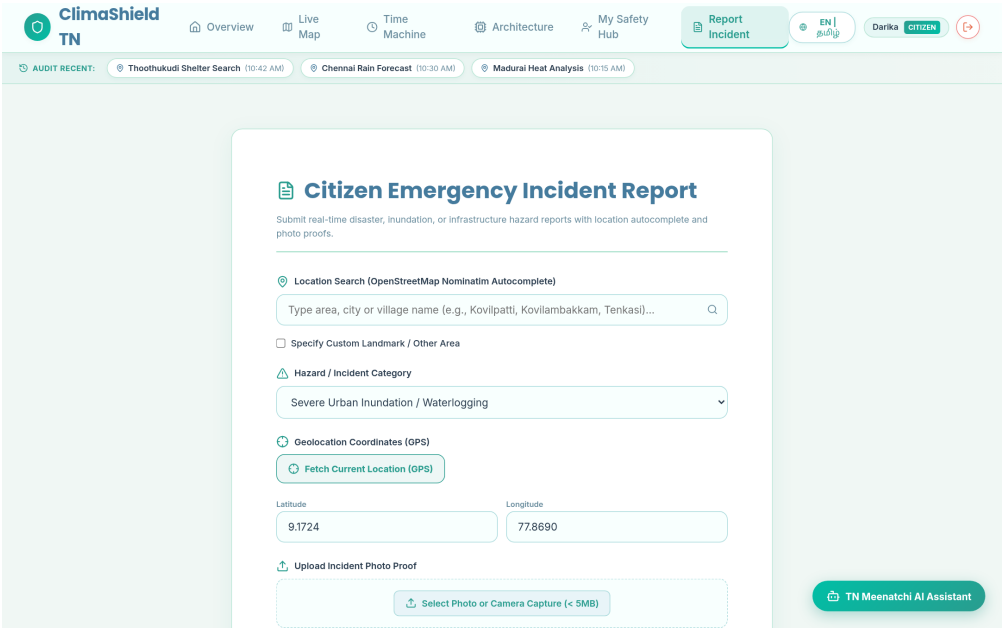


Fig 8.7 – Report Incident Form – Geo-tagged citizen hazard submission interface with location coordinates and incident details.

DISASTER TIME MACHINE

The Disaster Time Machine (/timeline) provides an interactive analytical environment for exploring 35 years of climate history across Tamil Nadu (1990–2024). Users can select any of the 8 core monitoring districts—including Chennai, Coimbatore, Madurai, Tiruchirappalli, Salem, Tirunelveli, Thoothukudi, and Cuddalore—and specify custom year ranges to generate dynamic time-series charts.

Powered by Recharts, the interface graphs annual mean temperatures, cumulative rainfall totals, and extreme heat days, revealing multi-decadal climate shifts. Interactive event markers spotlight historic disaster milestones—such as the 2015 Chennai Floods, Cyclone Vardah, Cyclone Gaja, and Cyclone Michaung—allowing researchers and policymakers to correlate observed meteorological anomalies with past disaster outcomes.

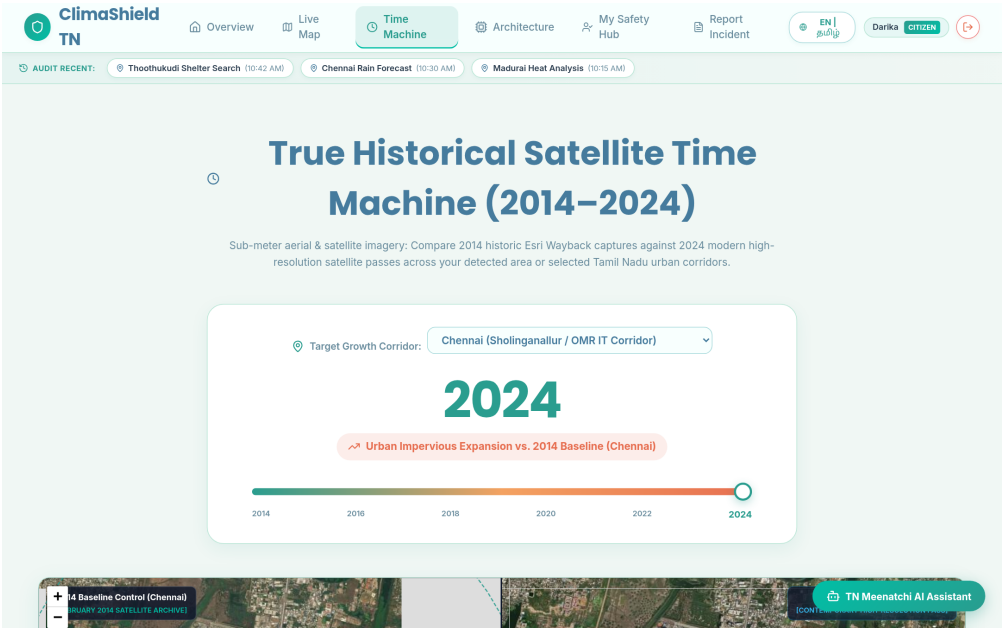


Fig 8.8 – Disaster Time Machine – Historical climate analysis and multi-decadal disaster timeline interface.

DISASTER IMPACT SIMULATOR

The Disaster Impact Simulator (/simulator) is a decision-support tool designed for emergency planners and disaster officers. It allows administrators to model hypothetical cyclone landfall trajectories along the Tamil Nadu coastline, adjusting parameters such as cyclone category (Category 1 to 5), wind velocity (up to 250 km/h), central atmospheric pressure, and projected rainfall intensity.

The simulation engine calculates dynamic flood inundation buffer zones around vulnerable river basins and low-lying coastal areas. It generates impact projections, including estimated affected population counts, high-risk household totals, required relief shelter capacities, and recommended emergency medical personnel deployments, enabling proactive resource positioning prior to storm landfall.

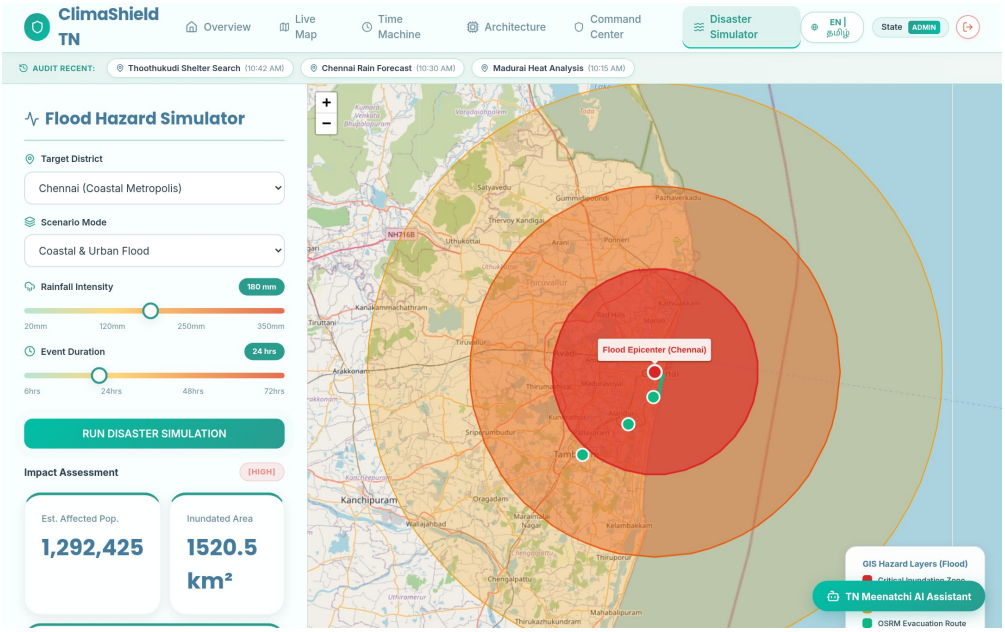


Fig 8.9 – Disaster Simulator – Interactive simulation engine for cyclone tracking and flood inundation impact modeling.

PAN-INDIA CLIMATE RISK OVERVIEW

The Pan-India Climate Overview (/india) contextualizes Tamil Nadu's climate exposure within the broader national landscape. Using D3.js and TopoJSON/GeoJSON projections, the page renders a vector choropleth map of India, color-coding all states and union territories according to composite climate vulnerability indices.

Hovering over or selecting any state opens a drill-down detail panel showing specific vulnerability metrics, including cyclone frequency, flood risk exposure, drought susceptibility, and heat stress indices. This comparative perspective helps state disaster managers understand inter-state climate patterns and coordinate interstate resource sharing during regional disaster events.

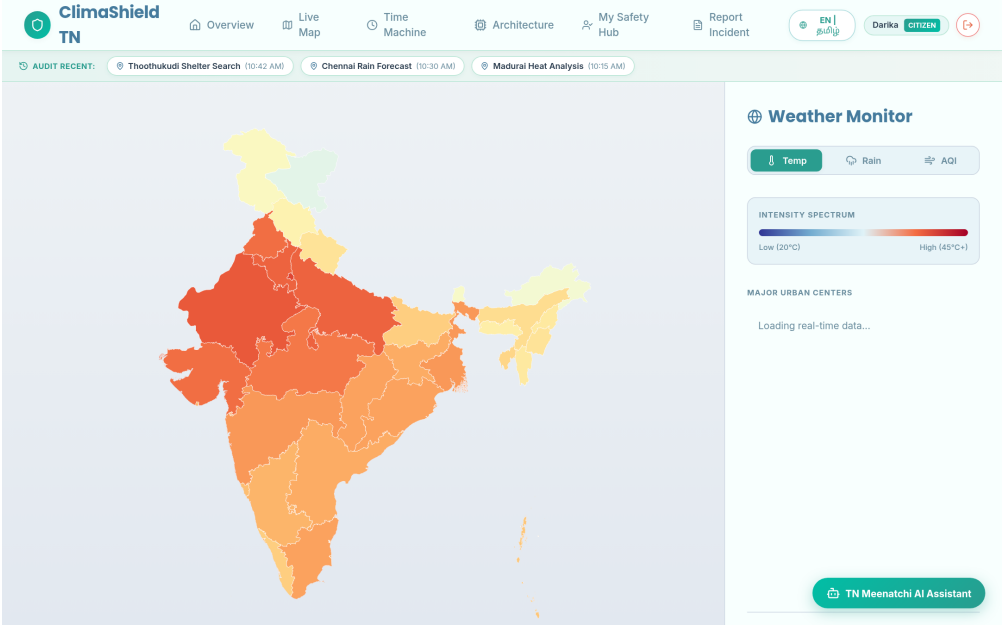


Fig 8.10 – Pan-India Climate Risk Overview – D3.js choropleth visualization of climate vulnerability across Indian states.

INCIDENT VERIFICATION LIFECYCLE

Every crowd-sourced incident report submitted to ClimaShield TN passes through a controlled state machine enforced by backend business logic. When a citizen submits a hazard observation, the record is initialized as PENDING_REVIEW. District disaster officers inspect incoming reports in their triage queue, verifying geotagged coordinates against satellite imagery and weather station feeds.

If confirmed, the officer transitions the status to VERIFIED_ACTIVE. This automatically updates the public live hazard map, broadcasting an active hazard pin and notifying nearby relief response teams. If a report is erroneous or duplicate, the officer marks it REJECTED, recording the dismissal in the audit ledger. Once emergency teams clear the hazard, the report transitions to RESOLVED, archiving the record.

FIGURE 8.11: CITIZEN INCIDENT VERIFICATION STATE MACHINE & WORKFLOW

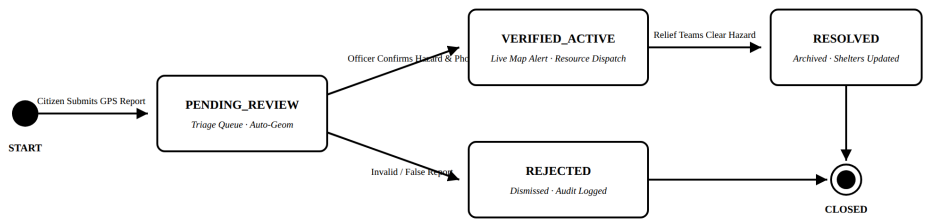


Fig 8.11 – Incident Lifecycle State Machine – Verification state transitions from citizen submission to resolution.

STATE DISASTER COMMAND CENTER

The State Disaster Command Center (/admin) serves as the primary operational hub for disaster officers and state administrators. The dashboard provides a real-time situational overview featuring platform-wide stat cards: total registered incidents, active verified hazards, statewide relief shelter capacity, and current overall shelter occupancy.

The central Incident Triage Queue displays all incoming citizen reports, showing hazard categories, submission timestamps, GPS coordinates, and attached evidence photos. Officers can review evidence and execute one-click status transitions (Verify, Reject, or Resolve). An interactive Shelter Management panel provides real-time occupancy updates, while the System Audit Log maintains an immutable chronological record of all administrative actions, ensuring complete operational transparency.

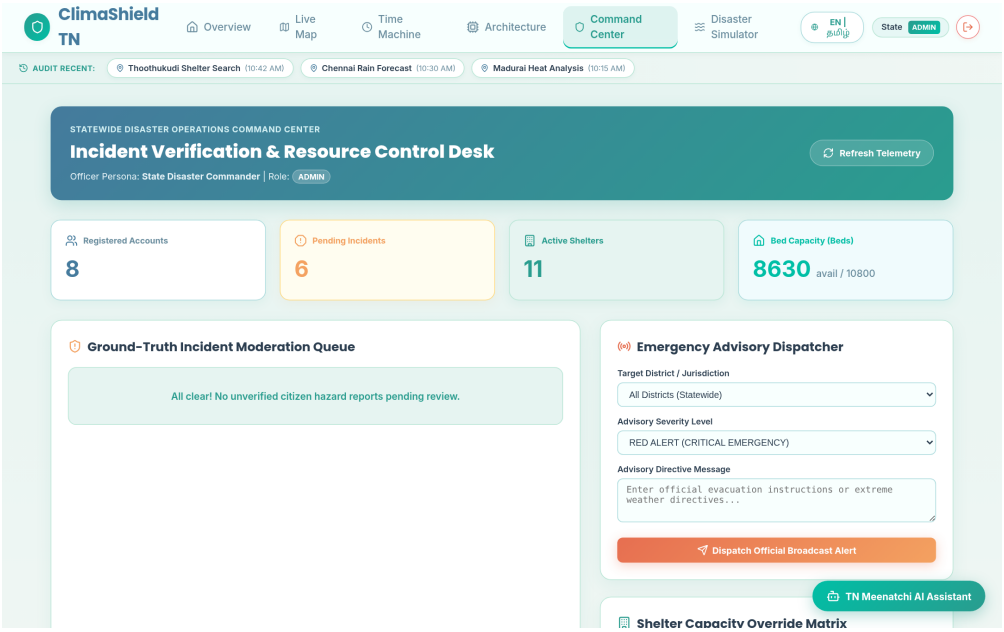


Fig 8.12 – Admin Command Center – State disaster command center with incident triage, shelter monitoring, and system metrics.

EMERGENCY AI CHATBOT ASSISTANT

The Emergency AI Chatbot is an interactive conversational widget accessible across the platform via a persistent floating action button. The assistant operates in both English and Tamil (தமிழ்), providing instant answers to common disaster queries. Citizens can ask about flood safety measures, cyclone preparation guidelines, the nearest functioning relief shelters, and district emergency telephone helplines.

The assistant processes natural-language inquiries through a dedicated backend intent parser (/api/v1/chat), matching queries against verified disaster response guidelines, meteorological definitions, and active shelter occupancies. By supporting localized Tamil vernacular, the chatbot ensures that critical emergency advisories are accessible to all citizens regardless of language barriers.

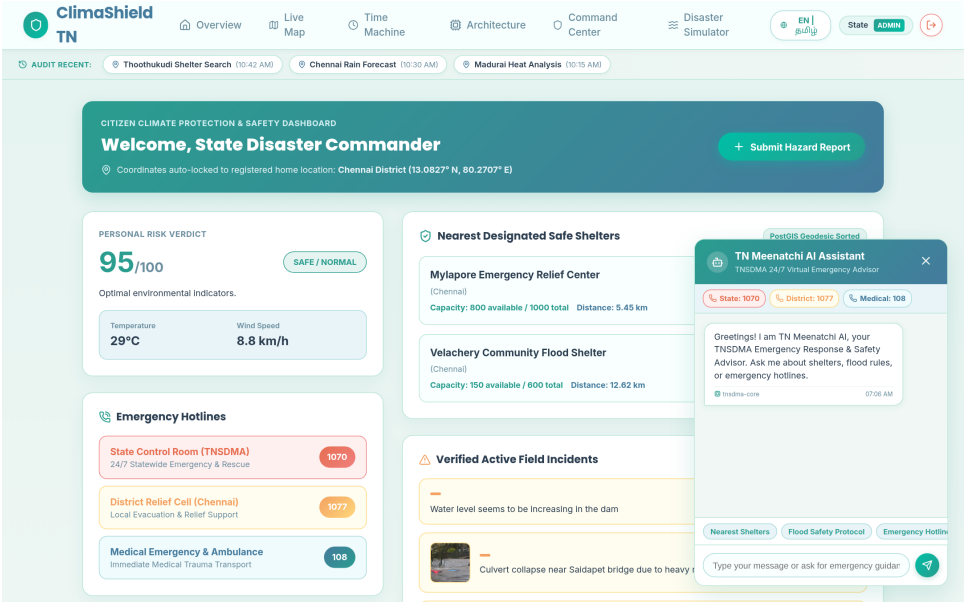


Fig 8.13 – Emergency AI Chatbot – Multilingual conversational assistant providing emergency guidance and shelter coordinates.

ARCHITECTURE & POSTGIS SPATIAL EXPLORER

The Architecture Hub (/architecture) provides an interactive system topology explorer for administrators and engineers. It details the multi-tier data pipeline, visualizes database entity relationships, and demonstrates live PostGIS geodesic SQL queries (such as ST_DWithin and ST_MakePoint), offering transparency into the underlying geospatial algorithms powering ClimaShield TN.

The interface highlights key architectural innovations, including the decoupling of vector Leaflet rendering from tile layers, the stateless verification pipeline of JWT sessions, and the performance characteristics of PostgreSQL R-Tree GiST spatial indexing, demonstrating engineering rigor across every layer of the modern web stack.

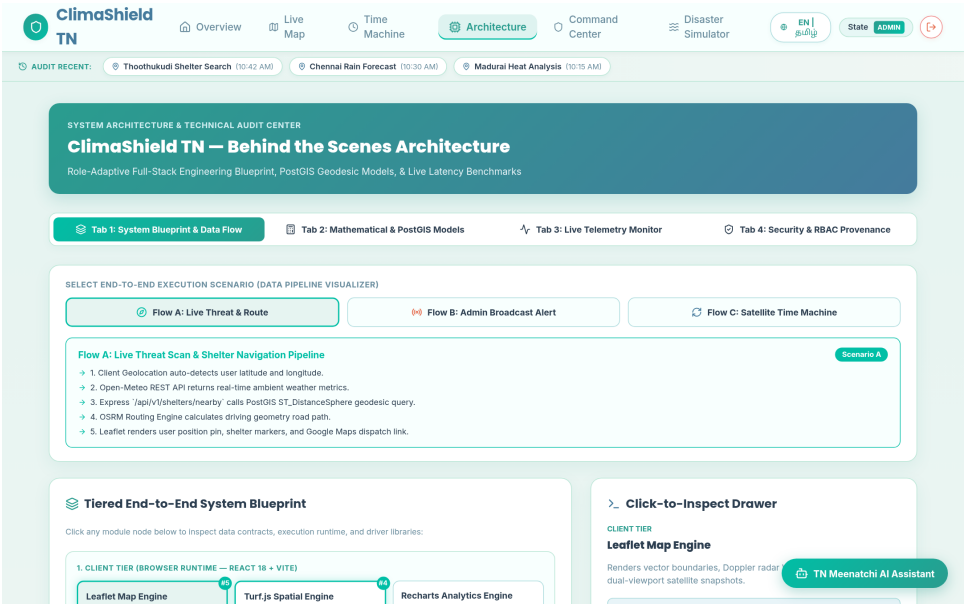


Fig 8.14 – Architecture Hub – Interactive system architecture and PostGIS query explorer.

8.3 SECURITY AND AUTHENTICATION

Security and data integrity are fundamental to ClimaShield TN, ensuring system availability during critical disaster scenarios. User passwords are encrypted using Bcrypt with an adaptive work factor (10 rounds) prior to database persistence. Following successful authentication, the server generates a cryptographically signed JSON Web Token (JWT) containing the user's identifier and assigned role. Tokens are transmitted via HTTP Authorization Bearer headers and validated on every protected API endpoint.

The platform enforces strict Role-Based Access Control (RBAC). Endpoints handling incident triage and alert broadcasting require ADMIN or DISTRICT_OFFICER privileges, while public citizens are restricted to personal profile updates and report submissions. All database queries utilize parameterized SQL statements through node-postgres, preventing SQL injection vulnerabilities. Cross-Origin Resource Sharing (CORS) policies restrict API access to verified client domains, while input sanitization pipelines validate all incoming GPS coordinates and payloads.

8.4 TESTING AND VERIFICATION

ClimaShield TN includes an automated integration test suite built with Jest and Supertest that validates backend functionality against the live database. The test suite exercises both core and edge-case execution paths across the application:

- • Health & System Status Checks: Validates that GET /api/health returns a 200 OK status and verifies that GET /api/v1/system/status confirms active database connectivity and complete data seeding (8 districts, 11 shelters, 3,360 climate records).
- • Historical Climate Series Validation: Verifies that GET /api/v1/climate/historical returns annual aggregates with validated numeric types for mean temperatures and rainfall across specified year intervals.
- • PostGIS Geodesic Distance Verification: Tests the GET /api/v1/shelters/nearby endpoint with coordinate parameters (lat=13.0827, lon=80.2707, radius_km=25), confirming candidate shelters are returned with validated numeric distance_km fields. Validates that requests missing coordinate parameters are rejected with a 400 Bad Request error.
- • Frontend Integration Testing: Verifies through Playwright end-to-end automation that all 10+ page routes render correctly under both Citizen and Admin sessions without runtime console exceptions.

CHAPTER 9
CONCLUSION

ClimaShield TN provides a modern, scalable, and practical technological solution for climate monitoring and disaster resilience in Tamil Nadu. By replacing fragmented, manual communication workflows with an integrated geospatial platform, the system establishes a direct digital link between public citizens experiencing emergencies on the ground and administrative command centers coordinating statewide relief operations. The enforcement of clear Role-Based Access Control ensures that citizens receive rapid shelter guidance, district officers maintain organized triage queues, and state administrators have comprehensive situational visibility across all 38 districts.

The integration of PostgreSQL PostGIS spatial indexing, the geodesic ST_DWithin proximity routing engine, the server-enforced incident verification state machine, the 35-year Disaster Time Machine, the interactive Disaster Impact Simulator, and the bilingual Emergency AI Assistant elevate ClimaShield TN beyond simple data tracking into an active decision-support system. The platform maintains accurate spatial records, accelerates emergency response times, mitigates misinformation through verified triage, and empowers community resilience.

While ClimaShield TN successfully meets all of its core design objectives, future enhancements could expand its operational capabilities. Planned extensions include integrating IoT water-level sensor telemetry deployed along the Adyar and Cooum rivers, incorporating synthetic aperture radar (SAR) satellite imagery for automated flood inundation mapping, and deploying an offline Progressive Web App (PWA) with background synchronization for environments with compromised cellular connectivity. Overall, ClimaShield TN demonstrates how modern web technologies can be harnessed to protect lives and build climate resilience across Tamil Nadu.

CHAPTER 10
REFERENCES

- • Tamil Nadu State Disaster Management Authority (TNSDMA). (2023). State Disaster Management Plan for Tamil Nadu. Revenue and Disaster Management Department, Government of Tamil Nadu.
- • Intergovernmental Panel on Climate Change (IPCC). (2023). Climate Change 2023: Synthesis Report. Contribution of Working Groups I, II and III to the Sixth Assessment Report of the IPCC, Geneva, Switzerland.
- • Open Geospatial Consortium (OGC). (2010). OpenGIS Implementation Standard for Geographic information - Simple feature access - Part 1: Common architecture. OGC Document 06-103r4.
- • Ramsey, P., & Neufeld, R. (2024). PostGIS Manual: Spatial and Geographic Objects for PostgreSQL (Version 3.4). Creative Commons Share Alike.
- • Facebook Open Source. (2024). React 18 Documentation: Building Reactive User Interfaces. Meta Platforms, Inc.
- • Vite Core Team. (2024). Vite: Next Generation Frontend Tooling. Available at: https://vite.dev/guide/
- • Modulz. (2023). Stitches: Near-Zero Runtime CSS-in-JS with Near-Zero Setup. Available at: https://stitches.dev/
- • Agafonkin, V. (2024). Leaflet: An Open-Source JavaScript Library for Mobile-Friendly Interactive Maps. Available at: https://leafletjs.com/
- • Bostock, M. (2023). D3: Data-Driven Documents. Available at: https://d3js.org/
- • OpenWeather Ltd. (2024). OpenWeatherMap Current Weather Data API Documentation. Available at: https://openweathermap.org/api
- • RainViewer Team. (2024). RainViewer Free Weather Radar API: Real-Time Precipitation Tile Services. Available at: https://www.rainviewer.com/api.html
- • Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT). RFC 7519, Internet Engineering Task Force (IETF).
- • Rescorla, E. (2018). The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446, Internet Engineering Task Force (IETF).
- • Indian Meteorological Department (IMD). (2023). Standard Operating Procedure for Cyclone Warning Services in India. Ministry of Earth Sciences, Government of India.